

Module 03: Feature Engineering I - Numerical Scaling and Skewness Transformations

Cybernauts 2026 Datathon Campaign

May 2026

Abstract

Raw numerical data can be deceptive. If you have an **Age** column (0–100) and an **Income** column (0–1,000,000), many machine learning algorithms will treat Income as 10,000 times more important simply because the numbers are larger. This note covers how to "level the playing field" through scaling and how to tame extreme outliers using logarithmic transformations.

Contents

1	The Scaling Problem: Why it Matters	2
2	Strategy 1: Standard Scaling (<i>Z</i>-Score)	2
3	Strategy 2: MinMax Scaling	2
4	Strategy 3: Log Transformation (Taming the Tail)	2
5	Practice Problems	2

1 The Scaling Problem: Why it Matters

Algorithms that calculate **distances** (like KNN, SVM, and K-Means) or use **gradients** (like Linear Regression and Neural Networks) are sensitive to the magnitude of numbers. If features aren't on the same scale, the model will "bully" the smaller features into irrelevance.

Tree-Based Exception

Models like **LightGBM**, **XGBoost**, and **Random Forests** are invariant to scaling. They only care about the *order* of values, not their magnitude. However, scaling is still a best practice if you plan to ensemble multiple model types.

2 Strategy 1: Standard Scaling (Z-Score)

Standard Scaling transforms your data so it has a **Mean of 0** and a **Standard Deviation of 1**.

$$z = \frac{x - \mu}{\sigma} \quad (1)$$

Use Case: Best for data that already follows a roughly Normal (Gaussian) distribution.

```
1 from sklearn.preprocessing import StandardScaler
2 scaler = StandardScaler()
3 df['Income_Scaled'] = scaler.fit_transform(df[['Income']])
4
```

3 Strategy 2: MinMax Scaling

This squishes all your values strictly into a range between **0** and **1**.

$$x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}} \quad (2)$$

Use Case: Essential for Neural Networks and algorithms where you need a bounded input range. *Warning:* This is highly sensitive to extreme outliers, as one massive value will squish all other data points toward zero.

4 Strategy 3: Log Transformation (Taming the Tail)

In the real world, values like **Transaction Amounts**, **City Populations**, or **User Activity** are often "Right-Skewed." They have a few massive outliers that create a "long tail."

The $\log(x + 1)$ Trick

A Log Transformation "pulls in" the extreme outliers and spreads out the smaller values, often turning a messy skewed distribution into a clean **Bell Curve**. We use $\log(x + 1)$ instead of $\log(x)$ to avoid errors if the data contains zeros.

```
1 # Taming a right-skewed distribution
2 df['Income_Log'] = np.log1p(df['Income'])
3
```

5 Practice Problems

P1: You are using a **K-Nearest Neighbors (KNN)** algorithm to group customers. Your features are **Years_Subscribed** (1–5) and **Total_Spend** (\$50–\$5,000). What will happen if you don't scale?

Answer: The distance calculation will be dominated by **Total_Spend**. The model will essentially ignore

how long a user has been subscribed because a difference of 4 years is tiny compared to a difference of \$4,000.

P2: Your column `Transaction_Amt` has a few values that are 100x larger than the median. Should you use MinMax Scaling?

Answer: No. MinMax Scaling is sensitive to outliers. Those 100x values will become 1.0, and almost all other data points will be squashed to values like 0.0001, losing all their detail. Use Log Transformation first.

P3: What is the standard deviation of a feature after applying `StandardScaler`?

Answer: 1.0 (and the mean will be 0.0).